

URL Shortener - Comparison of Approaches

1. Overview

Two main approaches for generating short URLs:

1. **Hash + Collision Resolution**
2. **Base62 Conversion**

Both achieve URL shortening but differ in **design, performance, and complexity**

2. Comparison Table

Feature	Hash + Collision Resolution	Base62 Conversion
Basic Idea	Hash long URL → truncate → resolve collisions	Convert unique ID → Base62 string
Input	Long URL directly	Unique numeric ID
Collision Handling	Required (collision possible)	Not required (unique ID ensures uniqueness)
Uniqueness Guarantee	No (needs checking + retry)	Yes (ID is unique)

Deterministic Mapping	Not guaranteed (due to collision handling)	Deterministic
Performance	Slower (DB lookup for collision check)	Faster (direct conversion)
Scalability	Moderate (collision overhead increases)	High (simple encoding)
Complexity	High (collision resolution logic needed)	Low (simple math conversion)
Dependency	Requires DB + possibly Bloom filter	Requires ID generator system
Length Control	Fixed (e.g., 7 chars after truncation)	Flexible (depends on ID size)

3. Key Differences Explained

Collision

- **Hash approach:**
 - Collisions are possible
 - Requires:
 - DB checks
 - Retry logic
- **Base62:**
 - No collision if ID is unique
 - Simpler system

Performance

- **Hash approach:**
 - Multiple DB queries possible
 - Slower under high load
- **Base62:**
 - Pure computation (division/modulo)
 - Much faster

Scalability

- **Hash approach:**
 - Performance degrades with scale (more collisions)
- **Base62:**
 - Scales easily with ID generator

System Dependency

- **Hash approach:**
 - Independent of ID generator
 - Works directly on URL
- **Base62:**
 - Requires **unique ID generator** (e.g., Snowflake)

4. When to Use What?

Hash + Collision Resolution

- When:
 - No ID generator available
 - Simpler setup required initially

Base62 Conversion

- When:

- High scale system
- Performance is critical
- Unique ID generator already exists

5. Final Takeaway

Base62 is generally preferred in real-world systems because:

- No collisions
- Better performance
- Simpler logic
- Highly scalable